

Comparison of benchmarks on a smart meter prototype

Jakob Fregin
Chair of Privacy and Security
TU Dresden
jakob.fregin@mailbox.tu-dresden.de

Abstract—The development and validation of smart meter software are limited by the restricted availability of proprietary hardware platforms and software environments. To address this challenge, the ARM cycle accounting emulator (ACAE) provides a hardware-independent prototyping platform with an instruction-level cycle accounting model for estimating embedded software execution performance. Previous validation using Dhrystone and CoreMark demonstrated consistent relative performance behavior but revealed substantial deviations in quantitative execution-time estimates. This work extends the validation of ACAE using the Embench benchmark suite to investigate workload-dependent timing behavior across a broader range of embedded applications. The results show that ACAE consistently reproduces the general performance characteristics of the evaluated workloads while systematically underestimating the required processor cycles. Across the Embench benchmarks, the observed performance deviations range from 20.4% to 55.9%, with an average deviation of 35.5%. The workload-based analysis indicates that the magnitude of the deviation depends on the computational characteristics of the evaluated applications, with compute-oriented workloads showing the largest contribution to the observed error. These results establish a more comprehensive validation baseline for ACAE and identify directions for future refinement of the cycle accounting model.

I. INTRODUCTION

The development of software for modern smart meters (SMs) is constrained by the proprietary nature of commercial metering platforms. Manufacturers typically provide only limited information about their hardware (HW) architectures and software environments, while access to development tool chains and representative HW is generally restricted. Consequently, the development, validation, and experimental evaluation of new SM algorithms are difficult to perform outside vendor-specific ecosystems. These limitations reduce reproducibility and hinder the rapid exploration of new software concepts for smart metering applications.

To address these challenges, our previous work introduced the ARM cycle accounting emulator (ACAE), a HW-independent SM prototyping platform that enables the development and evaluation of embedded SM software independently of proprietary HW [1]. At its core, ACAE employs a functional emulation of a representative SM microcontroller unit (MCU), providing a reproducible execution environment for embedded applications while abstracting from vendor-specific implementations. Beyond functional correctness, ACAE incorporates an instruction-level cycle accounting model that estimates processor execution time. This enables developers to evaluate

not only the functional behavior of software prototypes but also their expected computational performance on representative SM HW. Such performance estimates are particularly relevant because embedded SM algorithms may operate under strict execution-time and resource constraints.

The initial validation of ACAE assessed the accuracy of the cycle accounting model using the Dhrystone and CoreMark benchmarks [1]. Although the emulator consistently reproduced the relative performance characteristics of the target platform, the estimated execution times deviated substantially from measurements obtained on native HW. Specifically, execution time was underestimated by approximately 76% for Dhrystone and 51% for CoreMark. These results indicate that the current timing model captures the overall computational behavior of the representative processor but requires further refinement before it can provide reliable quantitative execution-time estimates.

However, Dhrystone and CoreMark each summarize processor performance using a single aggregate benchmark score. While these benchmarks are widely established for comparing embedded processors, they provide only limited insight into workload-dependent timing behavior. As a result, the previous validation cannot determine whether the observed timing deviations originate from systematic inaccuracies in the cycle model or from workload-specific characteristics of the evaluated benchmarks.

This work extends the validation methodology of ACAE by employing the Embench benchmark suite [2, 3], which comprises a diverse collection of representative embedded applications. By comparing the execution-time estimates produced by ACAE with measurements obtained on corresponding native HW across individual workloads, this work investigates the workload dependency of the current timing model and establishes a more comprehensive assessment of its accuracy.

The contributions of this work are threefold. First, the validation methodology of ACAE is extended from aggregate processor benchmarks to a diverse embedded benchmark suite. Second, workload-dependent timing deviations between the emulated and native platforms are characterized across representative embedded applications. Finally, the presented analysis identifies limitations of the current cycle accounting model and provides a foundation for its future refinement.

II. BACKGROUND

A. ACAE

ACAE was developed as a HW-independent SM prototyping platform that enables embedded software development independently of proprietary commercial SM HW [1]. Instead of reproducing an entire SM system, ACAE focuses on the processor functionality required for algorithm development, testing, and experimental evaluation. The underlying execution environment is based on a functional emulation of a representative ARM Cortex-M4-based SM MCU, allowing identical application software to execute emulated as well as on a native platform for validation.

To complement functional execution, ACAE incorporates an instruction-level cycle accounting model that estimates the execution time of embedded software. The resulting timing estimates provide an approximation of processor performance, enabling an initial assessment of whether software prototypes satisfy the execution-time constraints expected on representative SM HW. Previous validation demonstrated consistent functional execution and stable relative performance behavior but also revealed systematic underestimation of execution time when compared with native HW measurements obtained using Dhrystone and CoreMark [1]. This motivates a more comprehensive evaluation of the timing model across a broader range of embedded workloads.

B. MCU Model Validation

Performance validation of embedded processors and processor models commonly relies on standardized benchmark applications that provide representative workloads while enabling reproducible comparisons.

Historically, Dhrystone has been among the most widely used embedded processor benchmarks because of its simplicity and portability [4]. However, its synthetic integer-oriented workload and susceptibility to compiler optimizations have limited its representativeness for contemporary embedded software.

To address these shortcomings, the embedded microprocessor benchmark consortium (EEMBC) introduced CoreMark as a lightweight benchmark specifically targeting embedded processors [5]. CoreMark combines several representative computational kernels, including linked-list processing, matrix manipulation, state-machine execution, and cyclic redundancy check computation, into a single benchmark intended to better approximate practical embedded applications while remaining portable across architectures. Nevertheless, both Dhrystone and CoreMark ultimately summarize processor performance using a single aggregate score, limiting their usefulness for analyzing workload-dependent timing behavior.

Embench extends this methodology by providing a suite of 19 individual embedded applications instead of a single synthetic benchmark [2, 3]. The suite is derived primarily from the Bristol/Embecosm Embedded Benchmark Suite (BEEBS) benchmark collection [6], which incorporates representative workloads originating from MiBench [7], WCET [8], DSPstone [9], and Nettle [10]. The benchmarks target deeply

embedded systems with limited computational and memory resources while emphasizing portability and reproducibility. Unlike Dhrystone and CoreMark, Embench consists of a collection of individual applications that can be evaluated independently, enabling workload-specific performance analysis rather than relying solely on a single aggregate performance metric.

Embench was further characterized by Patterson et al. through a dynamic instruction analysis that classifies each benchmark according to the proportion of branching, memory, and compute operations executed during runtime [11]. Although this analysis was introduced as part of the Embench 2.0 publication, the reported workload characteristics also include the benchmark version evaluated in this work (1.0). The analysis was performed using a RISC-V implementation of the benchmark suite. Since these metrics describe the dynamic instruction mix rather than architecture-specific execution time, they provide a suitable characterization of the computational behavior of the evaluated applications. The resulting workload distributions are summarized in Table I.

Table I: Embench benchmarks and their distribution over branching, memory, and compute. [11]

Benchmark	Branching (%)	Memory (%)	Compute (%)
aha-mont64	10	1	89
crc32	14	14	72
cubic	14	16	69
edn	10	29	61
huffbench	23	26	51
matmult-int	12	38	50
minver	17	28	55
nbody	17	10	72
nettle-aes	2	20	78
nettle-sha256	1	14	84
nsichneu	45	54	1
picojpeg	11	28	61
qrdquo	15	20	65
sglib-combined	26	38	36
slre	27	31	42
st	16	11	72
statemate	14	72	13
ud	17	24	58
wikisort	20	38	42

For the validation of ACAE, these workload characteristics provide additional context beyond the execution-time measurements alone. Whereas the previous evaluation established the overall behavior of the cycle accounting model using Dhrystone and CoreMark, Embench enables observed timing deviations to be related to the computational characteristics of individual workloads. This facilitates investigating whether deviations are consistent across applications or associated with specific workload classes. Consequently, Embench provides a natural extension of the original validation methodology and forms the basis of the experimental evaluation presented in this work.

III. METHODOLOGY

A. Experimental Setup

The evaluation follows the same HW validation setup established in previous work [1]. In order to enable HW-independent

development of SM software, ACAE was developed based on a representative SM MCU model derived from publicly available device specifications. Searching for commercially available MCUs identified the ARM Cortex-M4 architecture as the closest match for the representative Smart Meter MCU model. This approach enables the reuse and adaptation of an existing ARM Cortex-M4 emulation model while preserving the relevant characteristics of the target SM platform.

Beyond functional execution, ACAE incorporates a cycle accounting model to approximate the execution time of embedded software. The cycle model estimates processor timing behavior by assigning estimated cycles to executed instructions and accounting for the resulting accumulated execution cycles. This timing information is exposed through a SM-equivalent cycle counter interface, which is available to benchmark applications in both the emulated and native execution environments. Consequently, identical benchmark instrumentation can be applied across both platforms, enabling a direct comparison of their execution behavior.

The evaluated platforms are summarized in Table II. The representative SM MCU model specifies an operating frequency and memory configuration of 100 MHz, 128 kB RAM, and 512 kB flash memory. A commercially available STM32F411CEU6 development board was selected as the native validation platform due to its matching ARM Cortex-M4 architecture and equivalent primary specifications. This alignment minimizes differences originating from processor frequency and memory capacity, allowing the evaluation to primarily assess the fidelity of the ACAE cycle model.

Table II: Evaluation platforms

	Processor	Frequency	RAM	Flash
ACAE	ARM Cortex-M4 model	100 MHz	128 kB	512 kB
Native	STM32F411CEU6	100 MHz	128 kB	512 kB

All benchmark applications were compiled using the same tool-chain and compiler configuration for both platforms. The benchmarks were built using `arm-none-eabi-gcc` with identical architecture-specific settings to ensure comparable generated code. Further implementation details of the benchmark environment and build configuration are provided in Section III-E.

B. Benchmark Integration

The benchmark evaluation is based on the Dhrystone and CoreMark benchmarks as well as the Embench benchmark suite. To enable execution in both ACAE and the native HW environment, all benchmarks were integrated into a common bare-metal evaluation framework. The required adaptations are limited to platform dependencies, timing interfaces, and result extraction, while the benchmark workloads and computational kernels remain unchanged. This approach ensures that both execution environments are evaluated using equivalent benchmark implementations.

The benchmarks were adapted to operate without an operating system by providing the required runtime dependencies

through the bare-metal support environment. Dhrystone requires basic standard library functionality, whereas CoreMark and Embench primarily rely on custom output and timing interfaces. For the native platform, the benchmark applications are linked against the required STM32F4 low-level support libraries, while ACAE provides the corresponding emulated runtime environment. In both cases, the same benchmark sources and evaluation flow are used, with differences restricted to platform-specific initialization and HW access.

To standardize benchmark execution and result collection, all benchmarks were integrated into a common build and execution framework. Each benchmark provides standardized build targets through its respective build configuration, which are invoked automatically by the evaluation harness. The harness performs the benchmark build and execution process, retrieves the generated output from either the native serial interface or the ACAE standard output stream, and extracts the reported performance values for further analysis.

The benchmark-specific result reporting was adapted to provide a unified output format consisting of a reported value and associated unit. For Dhrystone, the resulting performance score is calculated according to its original reference methodology based on the VAX 11/780 baseline. CoreMark reports the achieved iterations per second using its native performance metric. Embench reports the measured execution time in microseconds for the individual benchmarks. Although the reported metrics differ between benchmarks, the unified reporting interface enables automated evaluation while preserving the original benchmark-specific performance definitions.

C. Measurement Methodology

The benchmarks and benchmark suite considered in this work report their performance using different metrics. Dhrystone and CoreMark summarize performance as a benchmark score, where larger values indicate higher performance. In contrast, the individual benchmarks of the Embench suite report execution time. Consequently, different evaluation procedures are employed depending on the benchmark suite.

For Dhrystone and CoreMark, the benchmark score reported by each implementation is used directly for comparison. The relative deviation between ACAE and the native target platform is computed according to

$$\text{Deviation} = \frac{\text{Score}_{\text{ACAE}} - \text{Score}_{\text{native}}}{\text{Score}_{\text{native}}} \times 100\%. \quad (1)$$

Since larger benchmark scores correspond to higher performance, a positive deviation indicates that ACAE overestimates the computational performance of the target platform, whereas a negative deviation indicates an underestimation.

Unlike Dhrystone and CoreMark, Embench reports execution time for each benchmark individually rather than an aggregate performance score. For both the native platform and ACAE, execution time is derived from the measured processor cycle count. Within ACAE, the cycle counter interface emulates the behavior of the Cortex-M4 `DWT_CYCCNT` register, which exposes the accumulated number of HW-equivalent processor

cycles as a 32-bit counter. On the native platform, the corresponding HW register is accessed through the same bare-metal interface, ensuring identical measurement procedures across both execution environments. The measurement interface compensates for a single counter wrap-around, enabling correct runtime calculation for executions spanning one overflow event within the overall maximum of an unsigned 32-bit value.

The average execution time per benchmark iteration is calculated as

$$t_{\text{iter}} = \frac{N_{\text{cycles}}}{f_{\text{CPU}} N_{\text{iter}}}, \quad (2)$$

where N_{cycles} denotes the measured processor cycles, f_{CPU} the processor clock frequency, and N_{iter} the number of benchmark iterations. Runtime values are reported in μs .

To minimize measurement overhead, benchmark initialization is excluded from all runtime measurements. For the Embench benchmarks, two warm-up iterations are executed prior to measurement to eliminate initialization effects associated with cache population and benchmark setup. These warm-up iterations are not included in the reported results.

Each benchmark is executed with iteration counts ranging from 2^1 to 2^{12} . Since the underlying cycle counter is limited to 32 bits, measurements that would result in an irreversible counter overflow within a single execution are discarded. Consequently, the maximum evaluated iteration count is benchmark-dependent and may be limited to 2^{11} or 2^{10} for computationally intensive benchmarks.

Previous work analyzed the scaling behavior across the complete range of iteration counts in order to provide a broader characterization of the emulator's execution behavior. In contrast, the primary objective of this work is the comparative analysis of benchmark performance between ACAE and the native platform. Therefore, unless otherwise stated, the reported results are obtained using the highest iteration count supported by both platforms for the respective benchmark. This maximizes the measured execution interval, thereby reducing the relative influence of measurement overhead while maintaining comparability between both execution environments. One representative benchmark is additionally evaluated across the complete range of iteration counts to illustrate the influence of cache warm-up effects and to demonstrate that the selected measurement strategy captures the relevant execution characteristics without requiring an exhaustive evaluation of all iteration counts.

Because lower execution times correspond to higher performance, the runtime deviation is defined as

$$\text{Deviation} = \frac{\text{Runtime}_{\text{native}} - \text{Runtime}_{\text{ACAE}}}{\text{Runtime}_{\text{native}}} \times 100\%. \quad (3)$$

This formulation aligns the interpretation of runtime-based and score-based evaluations. In both cases, a positive deviation indicates that ACAE predicts higher performance than observed on the native HW. This corresponds to ACAE underestimating the execution time and, equivalently, the number of processor cycles required by the benchmark. Conversely, a negative deviation indicates that ACAE overestimates the execution time relative to the native platform.

D. Workload-Based Deviation Analysis

To complement the benchmark-wise comparison between ACAE and the native platform, the measured performance deviations of the Embench benchmarks were further analyzed with respect to their computational workload composition. For this purpose, the published workload characterization introduced in Section II-B was combined with the relative performance deviations obtained according to (3).

For each benchmark, the reported proportions of branching, memory, and compute operations were used as weighting factors describing the relative workload composition. Let d_i denote the measured relative performance deviation of benchmark i , and let $w_{i,\text{branch}}$, $w_{i,\text{memory}}$, and $w_{i,\text{compute}}$ denote the corresponding workload fractions reported by Embench. The workload-weighted component to the benchmark deviation is then computed as

$$d_{i,\text{branch}} = d_i w_{i,\text{branch}}, \quad (4)$$

$$d_{i,\text{memory}} = d_i w_{i,\text{memory}}, \quad (5)$$

$$d_{i,\text{compute}} = d_i w_{i,\text{compute}}, \quad (6)$$

with

$$d_i = d_{i,\text{branch}} + d_{i,\text{memory}} + d_{i,\text{compute}}. \quad (7)$$

The resulting weighted deviations were computed for all 19 Embench benchmarks using the automated evaluation framework and subsequently evaluated across the three workload categories. This analysis complements the benchmark-wise comparison by relating the observed timing deviations to the computational characteristics of the evaluated workloads, thereby enabling a workload-oriented assessment of the current ACAE cycle accounting model.

E. Reproducibility

All artifacts required to reproduce the presented experiments are provided through the accompanying repository [1]. The repository contains the ACAE emulator, the integrated benchmark implementations, the HW validation software, the evaluation framework, plotting scripts, and the source code of both the previous and the present work. The repository further includes setup scripts that install the required software dependencies, configure the tool-chain, and build the customized ARM QEMU version used by ACAE.

The experimental evaluation presented in this work was performed using the automated evaluation framework provided by the repository. Benchmark execution, data acquisition, and result collection are orchestrated through a Python-based evaluation harness, which compiles the benchmark applications, executes them on either ACAE or the native platform, and extracts the reported performance metrics into a common dataset. The figures presented in this work are generated directly from these datasets using the accompanying plotting script.

Benchmark applications are compiled separately for ACAE and the native platform to account for platform-specific startup

code and low-level HW support. Apart from these platform-dependent components, identical benchmark sources and compiler configurations are used throughout the evaluation. The benchmarks are compiled without compiler optimizations ($-O0$) to ensure consistency with the validation methodology established in previous work and to avoid introducing optimization-dependent effects that are not currently represented by the ACAE cycle model. Detailed compiler options and build configurations are documented within the repository.

Hardware validation requires an ARM Cortex-M4 development board with equivalent architectural characteristics. Communication with the evaluation framework is performed through a Universal Asynchronous Receiver-Transmitter (UART) interface. For the reference HW platform used in this work, an external ESP32-based UART relay provides the host communication interface. Alternative UART interfaces may be used, provided they expose equivalent serial communication capabilities.

IV. RESULTS

This section presents the validation results obtained for ACAE and the native HW platform. First, the influence of the benchmark iteration count is evaluated using one representative Embench benchmark to justify the measurement methodology. Subsequently, the relative performance deviations between ACAE and the native platform are presented for Dhrystone, CoreMark, and all Embench benchmarks.

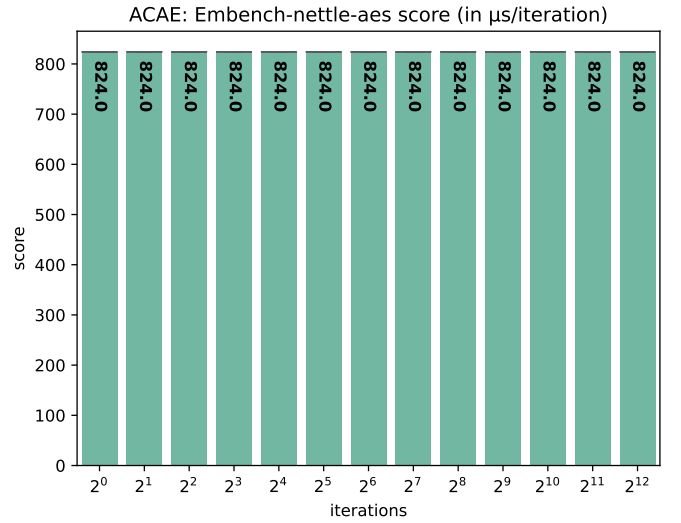
Following the methodology described in Section III-C, all benchmarks were evaluated for iteration counts from 2^1 to 2^{12} . Measurements exceeding the 32-bit cycle counter capacity were discarded, resulting in benchmark-dependent maximum iteration counts of 2^{10} , 2^{11} , or 2^{12} .

1) *Runtime Stability Across Iteration Counts*: The influence of the iteration count is illustrated using the Embench *nettle-aes* benchmark as a representative example.

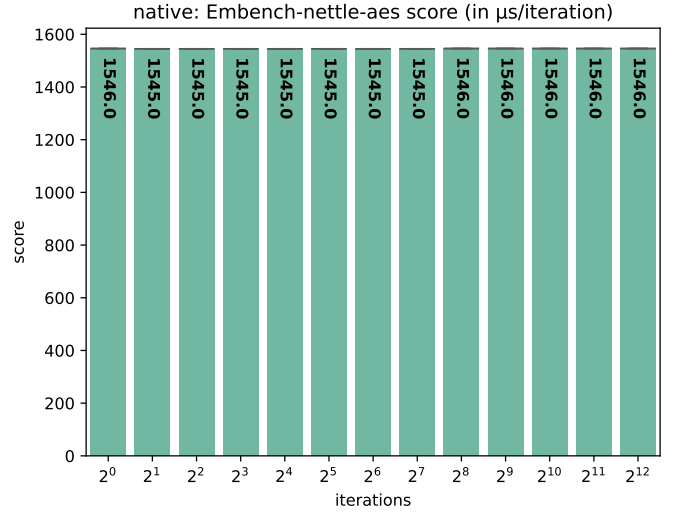
Figure 1 shows the average runtime per benchmark iteration, t_{iter} , for iteration counts ranging from 2^1 to 2^{12} . For ACAE, the measured runtime remains effectively constant at $824\mu\text{s}/\text{iteration}$, whereas the native platform remains approximately at $1546\mu\text{s}/\text{iteration}$. Across the evaluated iteration counts, the measured runtime per iteration remains nearly constant on both platforms, with only minor variations.

Figure 2 compares the average runtime per iteration obtained using the highest common iteration count of 2^{12} . For this benchmark, ACAE exhibits a relative performance deviation of 46.7% with respect to the native platform, corresponding to an underestimation of the required processor cycle count by the cycle accounting model.

The same evaluation procedure was applied to all Embench benchmarks. Since all benchmarks exhibit comparable runtime characteristics across the evaluated iteration counts, only the representative benchmark is shown. For the subsequent platform comparison, the highest common iteration count supported by both execution environments was used for each benchmark.



(a) Runtime on ACAE



(b) Runtime on native

Figure 1: Average runtime per iteration of the Embench *nettle-aes* benchmark for varying iteration counts.

2) *Benchmark Comparison*: Figure 3 summarizes the relative performance deviations between ACAE and the native platform for Dhrystone, CoreMark, and the complete Embench benchmark suite.

The previously evaluated Dhrystone and CoreMark benchmarks exhibit relative performance deviations of 73.8% and 48.7%, respectively. Across the 19 Embench benchmarks, the observed deviations range from 20.4% for *st* to 55.9% for *statemate*, with an arithmetic mean of 35.5% and a standard deviation of 8.5%. For all evaluated benchmarks, ACAE reports higher performance than the native platform, corresponding to a systematic underestimation of the required processor cycles by the cycle accounting model.

The CoreMark deviation of 48.7% lies within the range observed for the Embench benchmarks, whereas the Dhrystone

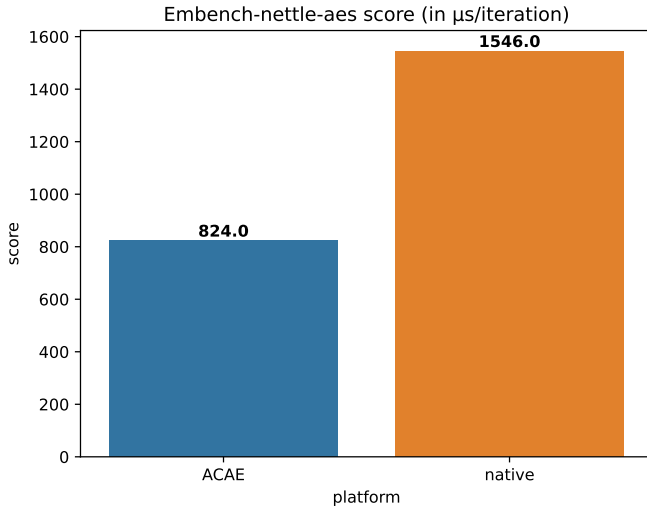


Figure 2: Comparison of the measured runtime per iteration for the Embench `nettle-aes` benchmark.

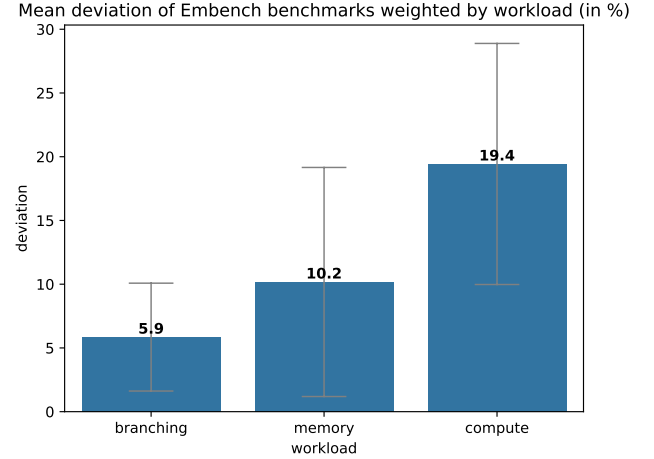


Figure 4: Aggregated workload-weighted performance deviations of the Embench benchmarks. Error bars indicate the standard deviation across all benchmarks.

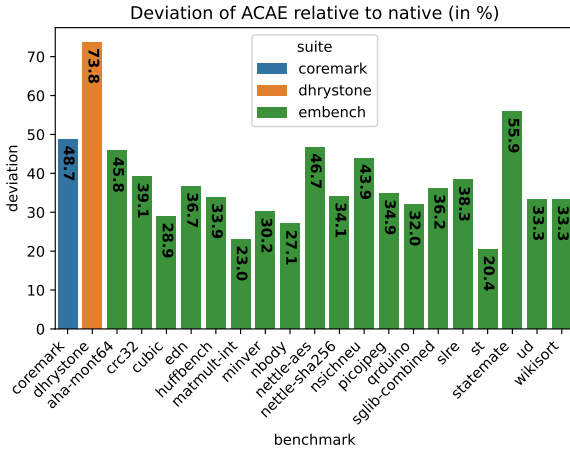


Figure 3: Relative performance deviation of ACAE with respect to the native platform for Dhrystone, CoreMark, and all Embench benchmarks.

deviation of 73.8% exceeds the maximum deviation observed within the Embench suite. Embench reports individual results for 19 workloads exposing considerably broader range than can be inferred from the two aggregate benchmarks alone.

3) *Workload-Based Deviation Analysis:* Figure 4 summarizes the workload-weighted performance deviations obtained using the methodology described in Section III-D. The relative performance deviation of each Embench benchmark was weighted according to its published workload composition and subsequently aggregated across all benchmarks.

The compute category exhibits the largest mean weighted deviation of 19.4%, followed by the memory category with 10.2% and the branching category with 5.9%. The corresponding standard deviations are 9.5%, 9.0%, and 4.2%, respectively.

Across the evaluated benchmark suite, the workload-weighted deviations therefore increase from branching to memory and compute. While all three workload categories are represented in the workload-weighted deviations between ACAE and the native platform, the compute category accounts for the largest share of the aggregated deviation.

V. DISCUSSION

A. Accuracy and General Behavior of the ACAE Timing Model

The evaluation presented in this work provides a more comprehensive assessment of the ACAE cycle accounting model compared with the previous validation based on Dhrystone and CoreMark. Across all evaluated benchmarks, ACAE consistently reports higher performance than the native platform, corresponding to an underestimation of the required processor cycles. The observed deviations are therefore not limited to individual benchmark characteristics but represent a systematic behavior of the current timing model.

The Embench evaluation extends the previous observations by covering a broader range of embedded workloads. While the previous evaluation indicated substantial deviations for Dhrystone and CoreMark, the individual Embench benchmarks reveal a more differentiated picture. The observed deviations range from 20.4% to 55.9%, with an average deviation of 35.5% and a standard deviation of 8.5%. This demonstrates that the timing accuracy of the current model depends to some extent on the executed workload. Nevertheless, the fact that all evaluated benchmarks exhibit a positive deviation indicates that the dominant limitation is a systematic underestimation of execution cycles rather than isolated benchmark-specific inaccuracies.

The comparison between aggregate benchmarks and the Embench suite further highlights the limitations of evaluating timing models using only single benchmark scores. The Dhrystone deviation of 73.8% exceeds the maximum deviation

observed within Embench, whereas the CoreMark deviation of 48.7% falls within the range of the individual workloads. Consequently, aggregate benchmarks can provide valuable initial indications of processor-level performance behavior but may not sufficiently represent the diversity of execution characteristics found in embedded applications.

The results therefore indicate that the current ACAE timing model successfully reproduces the general computational behavior of the target platform, while its quantitative execution-time prediction remains limited. For applications where relative performance comparisons are sufficient, the model provides useful information during early software development. However, applications requiring accurate execution-time guarantees, such as strict real-time validation, require further refinement of the underlying cycle accounting model.

B. Workload Dependency of Timing Deviations

The workload-wise evaluation demonstrates that the timing deviation varies between individual embedded applications. This variation confirms that the accuracy of the cycle accounting model is influenced by the executed instruction workload and cannot be fully characterized by a single benchmark value.

The Embench benchmark suite contains workloads with substantially different computational characteristics, ranging from compute-intensive applications such as `aha-mont64` and `nettle-sha256` to memory-intensive workloads such as `statemate`. The observed variation in deviations suggests that different instruction compositions interact differently with the timing assumptions of the current model. However, the systematic overestimation of performance across all workloads indicates that workload-specific effects contribute to the observed differences but are not the primary source of the overall timing error.

The workload-based analysis provides additional insight into these differences by relating the measured deviations to the published workload composition of the Embench benchmarks. The compute category exhibits the largest weighted contribution with a mean deviation of 19.4%, followed by memory-related operations with 10.2% and branching operations with 5.9%. These results indicate that inaccuracies related to computational and memory-related execution behavior may contribute substantially to the remaining timing error.

However, these results should be interpreted as an indication of workload dependency rather than a direct attribution of error sources. The workload composition represents the relative distribution of dynamic operations and does not directly describe the execution characteristics of the generated ARM Cortex-M4 binaries. Nevertheless, the observed correlation provides useful guidance for identifying aspects of the cycle accounting model that require further investigation.

C. Implications for Cycle Model Refinement

The workload analysis suggests several potential directions for improving the accuracy of the ACAE cycle accounting model. The comparatively large contribution of compute-oriented workloads indicates that the timing assumptions

associated with computational instructions may require refinement. Possible influencing factors include simplified instruction timing assumptions [1, 12], differences between nominal instruction cycles and actual processor execution behavior, or missing interactions between consecutive instructions. Consequently, a more fine-grained instruction accounting resolving heavier instructions such as division could lead to a significant decrease of the observed systematic cycle underestimation.

Similarly, the contribution of memory-oriented workloads indicates that memory access behavior may not yet be represented with sufficient accuracy. Although the evaluated platforms provide matching memory capacities and operating frequencies, the timing model currently abstracts the processor behavior through instruction-level cycle accounting. Effects such as memory access timing, flash wait states, or platform-specific memory characteristics may therefore contribute to the observed deviation.

In contrast, branching-related workloads show the smallest weighted deviation among the evaluated categories. This suggests that the current representation of branch-related execution behavior is comparatively closer to the native platform behavior. Nevertheless, the available analysis does not isolate individual instruction classes, and further validation using instruction-level performance measurements would be required to confirm the contribution of specific processor features.

Overall, the results indicate that improving the timing model should focus on reducing systematic timing underestimation while preserving the current ability to reproduce relative performance trends. Our results suggest that improving instruction class granularity of the cycle model could be beneficial, while reconstruction of branching and therefore pipeline effects may not be necessary. A refined model incorporating more detailed processor timing characteristics could improve the suitability of ACAE for quantitative execution-time estimation.

D. Limitations and Future Work

Several limitations should be considered when interpreting the presented results. Although the native validation platform was selected to closely match the representative MCU model, HW-specific implementation details such as memory subsystem behavior and device-specific timing characteristics may differ between both platforms. These differences represent an inherent challenge when validating an abstract processor model against physical HW and may contribute to the observed timing deviations.

First, although Embench provides a significantly broader workload representation than Dhrystone and CoreMark, it remains a benchmark-based evaluation rather than a complete characterization of all possible embedded application workloads. The presented results therefore characterize the accuracy of the ACAE timing model for representative processor workloads, while application-specific execution behavior may require additional validation.

Second, the evaluation intentionally uses an unoptimized compiler configuration to remain consistent with the previous

validation methodology and to avoid introducing optimization-dependent effects that are currently not represented by the timing model. Since compiler optimization can significantly influence the generated instruction sequence of embedded applications, future evaluations should investigate whether the timing accuracy of the cycle accounting model remains consistent across different optimization configurations.

Third, the workload characterization used in this work is based on the dynamic instruction analysis published for Embench. While this provides a useful description of the computational properties of the benchmarks, the reported instruction distributions were obtained for a RISC-V implementation and may differ from the generated ARMv7 binaries evaluated in this work. A future analysis based on architecture-specific instruction traces would provide a more direct characterization of the workload behavior. Since the ACAE cycle accounting model already relies on instruction-level instrumentation, extending the evaluation framework to collect architecture-specific workload characteristics would be a natural continuation of the presented methodology.

Future improvements of ACAE should therefore focus on refining the cycle accounting model using additional processor-level measurements and workload-specific analysis. The results presented in this work establish a more detailed baseline for such improvements by identifying the current accuracy limitations and providing a methodology for evaluating future model revisions.

VI. CONCLUSION

This work extended the validation of the ACAE timing model from aggregate processor benchmarks to a broader set of representative embedded workloads using the Embench benchmark suite. By comparing execution-time estimates obtained from ACAE with measurements on a corresponding native ARM Cortex-M4 platform, the evaluation provides a more detailed characterization of the current cycle accounting model and its workload dependency.

The results demonstrate that ACAE consistently reproduces the relative performance behavior of the evaluated workloads, while systematically underestimating the required processor cycles. Across the Embench benchmark suite, the observed performance deviations range from 20.4% to 55.9%, with an average deviation of 35.5%. The workload-based analysis further indicates that the magnitude of the deviation varies with the computational characteristics of the executed applications, with compute-oriented workloads contributing the largest share of the observed timing error. These results confirm that the current model provides useful performance estimates for comparative evaluation, while quantitative execution-time prediction requires further refinement.

Future improvements of ACAE should focus on refining the cycle accounting model through additional processor-level measurements and architecture-specific workload analysis. The workload-based evaluation suggests that improving the modelling of computational instructions may provide the greatest potential for reducing the remaining timing deviations, as

compute-oriented workloads showed the largest contribution to the observed deviation. This observation indicates a promising refinement direction but does not directly identify individual error sources. Since the current timing model already relies on instruction-level instrumentation, extending the evaluation framework with architecture-specific instruction traces provides a practical approach for further investigating and improving the model.

REFERENCES

- [1] Jakob Fregin. *ACAE*. <https://github.com/cheater78/acae>. Accessed: 2026-06-12.
- [2] Free and Open Source Silicon Foundation. *Embench: Open Benchmarks for Embedded Platforms*. <https://github.com/embench/embench-iot>. Accessed: 2026-07-20.
- [3] David Patterson et al. “Embench: Recruiting for the long overdue and deserved demise of Dhrystone as a benchmark for embedded computing”. In: *Computer Architecture Today Blog*, Jun 11 (2019).
- [4] Reinhold P. Weicker. “Dhrystone: a synthetic systems programming benchmark”. In: *Commun. ACM* 27.10 (Oct. 1984), pp. 1013–1030. ISSN: 0001-0782. DOI: 10.1145/358274.358283. URL: <https://doi.org/10.1145/358274.358283>.
- [5] Shay Gal-On and Markus Levy. “Exploring coremark a benchmark maximizing simplicity and efficacy”. In: *The Embedded Microprocessor Benchmark Consortium* 6.23 (2012), p. 87.
- [6] James Pallister, Simon Hollis, and Jeremy Bennett. “BEEBS: Open benchmarks for energy measurements on embedded platforms”. In: *arXiv preprint arXiv:1308.5174* (2013).
- [7] M.R. Guthaus et al. “MiBench: A free, commercially representative embedded benchmark suite”. In: *Proceedings of the Fourth Annual IEEE International Workshop on Workload Characterization. WWC-4 (Cat. No.01EX538)*. 2001, pp. 3–14. DOI: 10.1109/WWC.2001.990739.
- [8] Jan Gustafsson et al. “The Mälardalen WCET Benchmarks – Past, Present and Future”. In: ed. by Björn Lisper. Brussels, Belgium: OCG, July 2010, pp. 137–147.
- [9] RWTH Aachen University. *DSPstone - DSP Compiler and Processor Evaluation*. <https://www.ice.rwth-aachen.de/cms/ice/Forschung/Abgeschlossene-Projekte/~bjxhfy/DSPstone-DSP-Compiler-und-Prozessoreval/lidx/1/>. Accessed: 2026-07-25.
- [10] Niels Möller. *Nettle - a low-level cryptographic library*. <https://www.lysator.liu.se/~nisse/nettle/>. Accessed: 2026-07-26.
- [11] David Patterson et al. “Embench IOT 2.0 and DSP 1.0: Modern Embedded Computing Benchmarks”. In: *Computer* 58.5 (2025), pp. 37–47. DOI: 10.1109/MC.2024.3511352.

- [12] Arm Limited. *Arm Cortex-M4 Processor Technical Reference Manual*. <https://developer.arm.com/documentation/100166/0001>. Accessed: 2026-05-01.